

Syntax-directed Translations

1. What is syntax-directed translation? Explain.

Ans: **Syntax-directed translation (SDT)** is an extension of context-free grammar (CFG) which acts as a notational framework for the generation of intermediate code. It embeds program fragments (called **semantic actions**) within the production bodies. For example, as in

$$A \rightarrow A_1 * B \{ \text{print } '**' \}$$

In SDT, by convention, we use curly braces to enclose semantic actions. If the grammar symbols contain curly braces, then we enclose them in single quotes as `'{'` and `'}'`. The order of execution of semantic actions is determined by the position of a semantic action in a production body. Here, in the above example, the action occurs at the end, that is, after all the grammar symbols. However, semantic actions can occur at any position in the production body.

In syntax-directed translation, we pass the token stream as input, build the parse tree for the input token stream, and then evaluate the semantic rules at the nodes of parse tree by traversing the parse tree (as shown in Figure 6.1). The evaluation of semantic rules by syntax-directed translation may generate an intermediate code, save information in the symbol table, perform semantic checking, issue error messages, or perform some other activities.



Figure 6.1 Conceptual View of Syntax Directed Translation

2. Explain syntax-directed definition.

Ans: **Syntax-directed definition (SDD)** associates attributes with the grammar symbols and semantic rules with each grammar production. In other words, SDD is a context-free grammar with the information to control the semantic analysis and translation. These grammars are also called **attribute grammars**. A grammar is augmented by associating attributes with each grammar symbol that describes

its properties, and the rules are associated with productions. If A is a symbol and i is one of its attributes, then we can write $A.i$ to denote the value of i at a particular parse tree node. An attribute has a name and an associated value which can be a string, a number, a type, a memory location, or an assigned register. The attribute value may depend on its child nodes or siblings or its parent node information.

The syntax-directed definition is partitioned into two subsets, namely, *synthesized* and *inherited* attributes. Semantic rules are used to set up dependencies between attributes that will be represented by a graph. An evaluation order for the semantic rules can be derived from the dependency graph. The values of the attributes at the nodes in parse tree are defined by the evaluation of the semantic rules.

3. Compare syntax-directed translation and syntax-directed definition.

Ans: Translations are carried out during parsing and the order in which the semantic rules are evaluated by the parser must be explicitly specified. Hence, instead of using the syntax-directed definitions, we use syntax-directed translation schemes to specify the translations. Syntax-directed definitions are more abstract specifications for translations; therefore, they hide many implementation details, freeing the user from having to explicitly specify the order in which translation takes place. Whereas, the syntax-directed translation schemes indicate the order in which semantic rules are evaluated, allowing some implementation details to be specified. Syntax-directed definitions are more readable and therefore, are useful for specification, whereas translation schemes are very much efficient and, therefore, are useful for implementation.

4. What are synthesized and inherited attributes? How semantic rules are attached to the productions?

Ans: Synthesized attributes: A synthesized attribute for a non-terminal A at a parse tree node Γ is defined by a semantic rule associated with the production at Γ and the production must have A as its head. A synthesized attribute at node Γ has its value defined only in terms of attribute values of its child nodes and Γ itself.

For example, consider the grammar for a desk calculator.

$$\begin{aligned} E &\rightarrow M \\ M &\rightarrow M' + P \\ M &\rightarrow P \\ P &\rightarrow P' * R \\ P &\rightarrow R \\ R &\rightarrow (M) \\ R &\rightarrow \text{digit} \end{aligned}$$

The given grammar describes arithmetic expressions with operators $+$ and $*$. In SDD, each of the non-terminals has a single synthesized attribute, called `val`, and `digit` has a synthesized attribute `lexval` (the lexical value for the digit which is an integer value returned by the lexical analyzer).

So, semantic rules for this grammar can be written as given in Figure 6.2.

In production rule 1, $E \rightarrow M$, sets $E.val$ to $M.val$. The production rule 2, $M \rightarrow M' + P$, computes the `val` attribute for the head M as the sum of the values at its child nodes M' and P . Production 3, 5, and 6 are same as that of production rule 2. Production rule 7 gives $R.val$, the numerical value of the token `digit` which is returned by the lexical analyzer.

Since the entire attribute values of the symbols as head are defined in terms of attributes at their child nodes, it means all attributes involved are synthesized attributes and the corresponding SDD is known as **S-attributed definition**.

PRODUCTION	SEMANTIC RULES
1. $E \rightarrow M$	$E.val = M.val$
2. $M \rightarrow M' + P$	$M.val = M'.val + P.val$
3. $M \rightarrow P$	$M.val = P.val$
4. $P \rightarrow P' * R$	$P.val = P'.val * R.val$
5. $P \rightarrow R$	$P.val = R.val$
6. $R \rightarrow (M)$	$R.val = M.val$
7. $R \rightarrow \text{digit}$	$R.val = \text{digit.lexval}$

Figure 6.2 Syntax Directed Definition of a Simple Desk Calculator

Inherited attributes: An inherited attribute for a non-terminal A at a parse tree node I is defined by a semantic rule associated with the production at the parent of I , and the production must have A as a symbol in its body. The value of an inherited attribute at node I can only be defined in terms of attribute values of I 's parents, I 's siblings, and I itself. Inherited attributes are convenient for expressing the dependence of a programming language construct on the context on which it appears. For example, an inherited attribute can be used to keep track of whether an identifier appears on the left side or right side of an assignment operator in order to determine whether the address or the value of the identifier is required.

For example, consider the following grammar:

```

E → AB
A → int
B → B', id
B → id

```

The syntax-directed definitions that use inherited attributes can be written as:

PRODUCTION	SEMANTIC RULES
1. $E \rightarrow AB$	$B.inh = A.type$
2. $A \rightarrow \text{int}$	$A.type = \text{integer}$
3. $B \rightarrow B', id$	$B'.inh = B.inh$ $\text{enter} (id.print, B.inh)$
4. $B \rightarrow id$	$\text{enter} (id.print, B.inh)$

Figure 6.3 Syntax Directed Definition for Inherited Attributes

The non-terminal symbol A in the productions has the synthesized attribute type whose value can be obtained by the keyword in the declaration. The semantic rule $B.inh = A.type$ sets inherited attributes $B.inh$ to the type in the declaration.

The parse tree with the attributes values at the parse tree nodes, for an input string $\text{int } id_1, id_2, id_3$ is shown in Figure 6.4. The type of identifiers $id_1, id_2,$ and id_3 is determined by the value of $B.inh$ at the three B nodes. These values are obtained by computing the value of the attribute $A.type$ at the left child of the root and then evaluating $B.inh$ in top-down at the three B nodes in the right subtree of the root. We also call the procedure enter at each B node to insert into the symbol table, where the identifier at the right child of this node is of type int .

Ans: A **dependency graph** represents the flow of information between the attribute instances in a parse tree. It is used to depict the interdependencies among the inherited and synthesized attributes at the nodes in a parse tree. When the value of an attribute needs to compute value of another attribute, then an edge from first attribute instance to another is created to indicate the dependency among the attribute instances. That is, if an attribute x at a node in a parse tree depends on an attribute y , then the semantic rule for y at that node must be evaluated before the evaluation of the semantic rule that defines x .

To construct a dependency graph for a parse tree, we first make each semantic rule in the form $x := f(y_1, y_2, y_3, \dots, y_k)$ by introducing a dummy synthesized attribute x for each semantic rule that consists of a procedure call. We then create a node for each attribute in the graph and an edge from node y to the node x , if attribute x depends on attribute y .

The algorithm for constructing the dependency graph for a given parse tree is shown in Figure 6.6.

```

For each node  $n$  in the parse tree do
  Begin
    For each attribute  $b$  of the grammar symbol at  $n$  do
      Begin
        construct a node in the dependency graph for  $b$ 
      End
    End
  For each node  $n$  in the parse tree do
    Begin
      For each semantic rule  $x := f(y_1, y_2, \dots, y_k)$  associated with
      the production used at  $n$  do
        Begin
          For  $i := 1$  to  $k$  do
            Begin
              create an edge from the node for  $y_i$  to the
              node for  $x$ 
            End
          End
        End
      End
    End
  End

```

Figure 6.6 Algorithm to Construct the Dependency Graph

7. Construct a dependency graph for the input string $7 + 8$, by considering the following grammar:

$A \rightarrow BA'$
 $A' \rightarrow + BA'_1 \mid \in$
 $B \rightarrow \text{digit}$

Ans: The semantic rules for the given grammar productions can be written as:

The SDD in Figure 6.7 are used to compute $7 + 8$, and the parsing begins with the production $A \rightarrow BA'$. Here, B generates the digit 7, but the operator $+$ is generated by A' . As the left operand 7 appears in a different subtree of the parse tree from $+$, so an inherited attribute is used to pass the operand to the operator.

PRODUCTION	SEMANTIC RULES
1. $A \rightarrow BA'$	$A'.inh = B.val$ $A.val = A'.syn$
2. $A' \rightarrow + BA_1'$	$A_1'.inh = A'.inh + B.val$ $A'.syn = A_1'.syn$
3. $A' \rightarrow \epsilon$	$A'.syn = A'.inh$
4. $B \rightarrow \text{digit}$	$B.val = \text{digit.lexval}$

Figure 6.7 Syntax Directed Definition for the Given Grammar

A synthesized attribute `val` is used for each of the non-terminals `A` and `B` and a synthesized attribute `lexval` is used for the terminal `digit`. The non-terminal `A'` has two attributes, an inherited attribute `inh` and a synthesized attribute `syn`.

In the given string `7 + 8`, the operator `+` inherits `7` as shown in Figure 6.8.

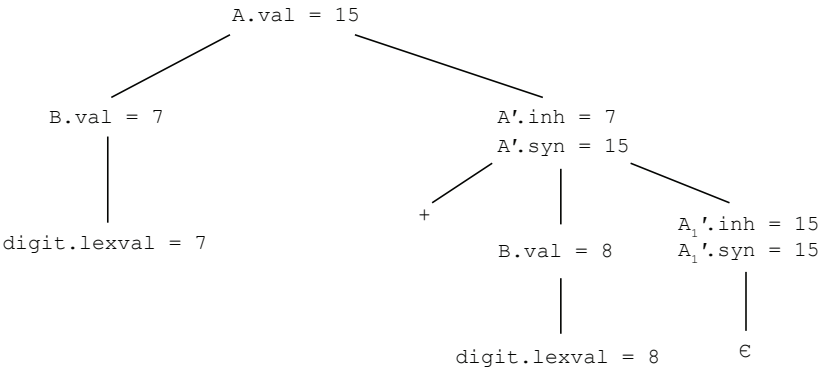


Figure 6.8 Annotated Parse Tree for `7 + 8`

We use this parse tree to construct a dependency graph and the corresponding dependency graph is shown in Figure 6.9. The nodes in the dependency graph are numbered from 1 to 9, and they correspond to the attributes in the annotated parse tree.

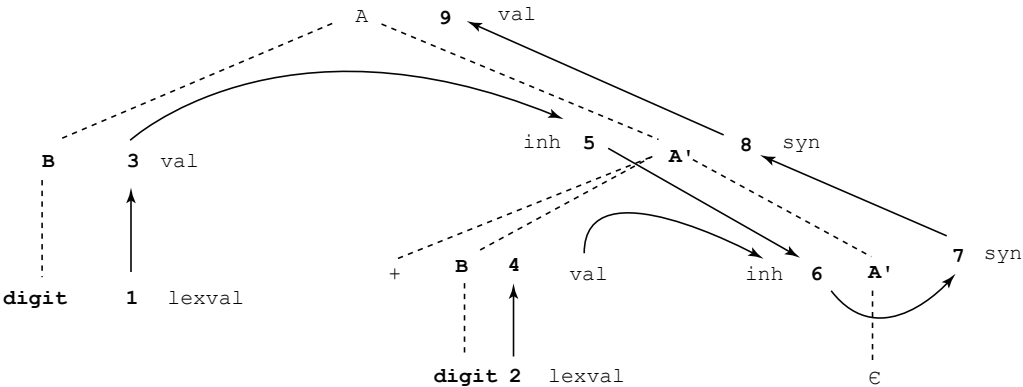


Figure 6.9 Dependency Graph for the Annotated Parse Tree of Figure 6.8

The two leaves `digit` are associated with attribute `lexval` and are represented by nodes 1 and 2. The two nodes labeled `B` are associated with the attribute `val` and are represented by the nodes 3 and 4. The edges from node 1 to node 3 and from node 2 to node 4 use the semantic rule that defines `B.val` in terms of `digit.lexval`.

Each occurrence of non-terminal A' is associated with the inherited attribute $A'.inh$, and are represented by nodes 5 and 6. The edge from 3 to 5 is due to the rule $A'.syn = A'.inh$. The edge from node 5 to node 6 is for $A'.inh$ and from node 4 to node 6 is for `B.val`, because these values are added to calculate the attribute `inh` at node 6.

The synthesized attribute `syn` associated with the occurrences of A' is represented by nodes 7 and 8. The edge from node 6 to node 7 is due to the semantic rule $A'.syn = A'.inh$ associated with the production 3. The edge from node 7 to node 8 is due to the semantic rule associated with production 2. Node 9 represents the attribute `A.val`. The edge from node 8 to node 9 is due to the semantic rule, $A.val = A'.syn$, associated with production 1.

8. What are S-attributed definitions and L-attributed definitions?

Ans: S-Attributed definitions: A syntax-directed translation is called S-attributed if all its attributes are synthesized. The attributes of an S-attributed SDD can be evaluated using the bottom-up approach of the traversal of the parse tree in which the attributes of the parse tree are evaluated by performing a post-order traversal of the parse tree. In post-order traversal, we evaluate the attributes at a node N when the traversal leaves N for the last time. That is, we, apply the post-order function given in Figure 6.10 to the root of the parse tree.

```

postorder(N)
Begin
    For each child  $C$  of  $N$ , from left to right
        postorder(C);
    Evaluate the attributes associated with node  $N$ 
End

```

Figure 6.10 Algorithm for Computing Postorder Function

L-attributed definition: An L-attributed definition is another class of SDD in which the dependency graph edges can only go from left to right and not vice-versa. Each attribute in L-attributed definition must be either synthesized or inherited. If the attributes are inherited, then they must follow some rules. Assume we have a production $A \rightarrow Y_1 Y_2 \dots Y_n$, and $Y_i.a$ is an inherited attribute evaluated by a rule associated with the given production. Then the rule may only use:

- Inherited attributes that are related with the head A .
- The attributes (either synthesized or inherited) that are related with the occurrences of the symbols $Y_1, Y_2, \dots, Y_{i-1}$ (that is, the symbols to the left of Y_i in the production).
- Synthesized or inherited attributes related with Y_i in such a way that there are no cycles in the dependency graph formed by the attribute of Y_i .

For example, consider the syntax-directed definitions given in Figure 6.7. To prove that the SDD in Figure 6.7 is L-attributed, consider the semantic rules for inherited attributes as shown in Figure 6.11.

The syntax-directed definition above is an example of the L-attributed definition, because the inherited attribute $A'.inh$ using only `B.val`, and B is appearing to the left of A' in the production $A \rightarrow BA'$. Similarly, the inherited attribute $A_1'.inh$ in the second rule is defined using the inherited attribute

PRODUCTION	SEMANTIC RULES
$A \rightarrow BA'$	$A'.inh = B.Val$
$A \rightarrow +BA_1'$	$A_1'.inh = A'.inh + B.val$

Figure 6.11 Syntax Directed Definition of Inherited Attributes.

$A'.inh$ related with the head, and $B.val$, where B appears on the left of A_1' in the production $A' \rightarrow + BA_1'$. In both cases, the rules for L-attributed definitions are followed, and the remaining attributes are synthesized (as shown in Figure 6.7). Therefore, this SDD is L-attributed.

9. Discuss the applications of syntax-directed translation.

Ans: Syntax-directed translations are applied in the following techniques:

- ❑ **Construction of syntax tree:** Syntax tree is used as an intermediate representation in some compilers and, hence, a common form of SDD converts its input string into the syntax tree. To construct the syntax tree for expressions, we can use either an S-attributed or an L-attributed definition. The S-attributed definitions are suitable to use in bottom-up parsing, whereas the L-attributed definitions are suitable to use in top-down parsing.
- ❑ **Type checking:** Type checking is used to catch errors in the program by checking type of each variable, constant, functions, and expressions. Type checking eliminates the need for dynamic checking for type errors.
- ❑ **Intermediate code generation:** Intermediate codes are machine-dependent codes and are close to the machine instructions. Syntax-directed translation, postfix notation, and syntax tree can be used as an intermediate code.

10. What is a syntax tree? Explain the procedure for constructing a syntax tree with the help of an example.

Ans: A **syntax tree** or an **abstract syntax tree (AST)** is a tree representation showing the syntactic structure of the source program. It is a compressed form of a parse tree representing the hierarchical construction of a program, where the nodes represent operators and the children of any node represent the operands that are to be operated by that operator. For example, the syntax tree for the expression $p * (q + r) / s$ is shown as in Figure 6.12.

The construction of syntax tree for an expression can be considered as the translation of the expression into post-fix form.

The subtrees are constructed for the subexpressions by creating a node for each operator and operand. The children of an operator node are the roots of the nodes representing the subexpressions constituting the operands of that operator.

The nodes of a syntax tree are implemented as objects having several fields. Each node is labeled by the `op` field, which is often called the label of the node. When used for translation, the nodes in a syntax tree may have additional fields to hold the values of attributes attached to the node, which are as follows:

- ❑ For a leaf node, an additional field is required to hold the lexical value of the leaf. A constructor function `Make-leaf(num, val)` or `Make-leaf(id, entry)` is used to create a leaf object.
- ❑ If the node is an interior node, a constructor function `Make-node(op, left, right)` is used to create an object with first field `op` and two additional fields for its left and right children.

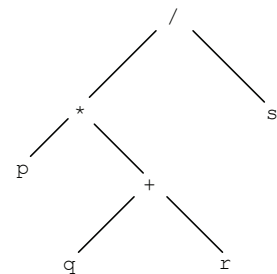


Figure 6.12 A Simple (Abstract) Syntax Tree

For example, consider the expression $x - 7 + z$. In this, we need the following functions to create the nodes of syntax trees for expressions with binary operators.

- `Make-node (op, left, right)` creates an operator node with label `op` and two fields containing pointers to left and right children.
- `Make-leaf(id, entry)` creates an identifier node with label `id` and a field containing `entry`, a pointer to the symbol table entry for the identifier.
- `Make-leaf(num, val)` creates a number node with label `num` and a field containing `val`, the value of the number.

Consider the S-attributed definition shown in Figure 6.13 that constructs syntax tree for the expressions involving only binary operators $+$ and $-$. All the non-terminals have only one synthesized attribute `node` that represents a node of the syntax tree.

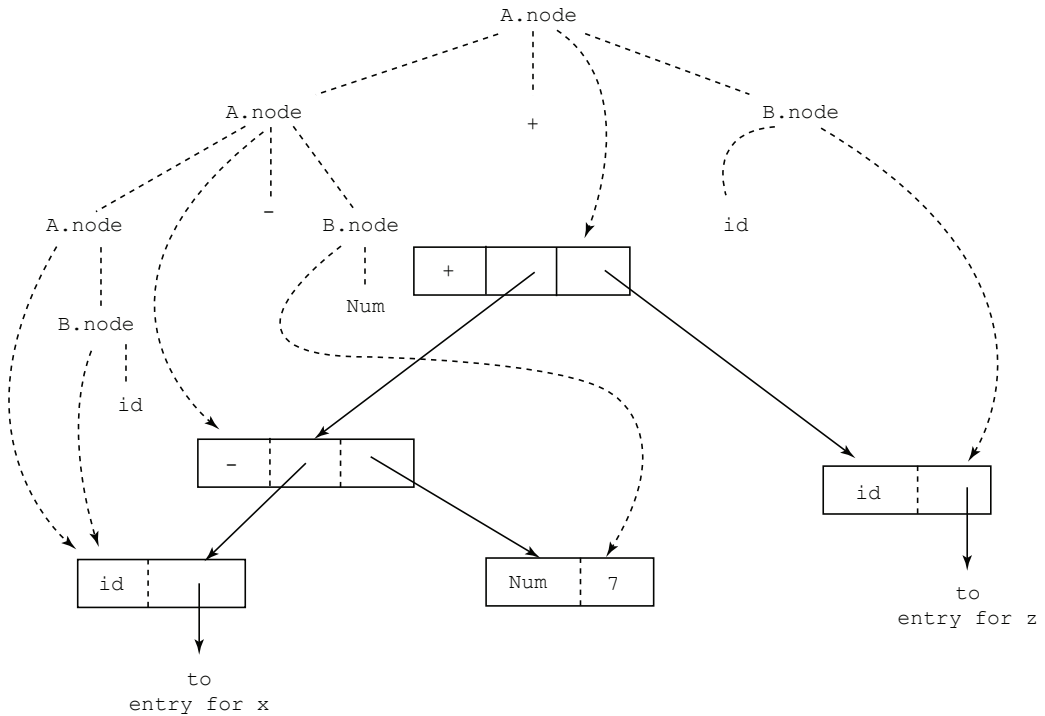


Figure 6.13 Syntax Tree for $x - 7 + z$

To create the syntax tree for the expression $x - 7 + z$, we need a sequence of function calls, where p_1, p_2, p_3, p_4, p_5 are the pointers to nodes, and entry x and entry z are pointers to the symbol table entries for identifiers x and z , respectively.

1. $p_1 := \text{Make-leaf}(\text{id}, \text{entry } x);$
2. $p_2 := \text{Make-leaf}(\text{num}, 7);$
3. $p_3 := \text{Make-leaf}('-', p_1, p_2);$
4. $p_4 := \text{Make-leaf}(\text{id}, \text{entry } z);$
5. $p_5 := \text{Make-leaf}('+', p_3, p_4);$

The tree is constructed using bottom-up approach. The function calls `Make-node(id, entry x)` and `Make-node(num, 7)` construct the leaves for `x` and `7`, the pointers to these nodes are saved using `p1` and `p2`.

The function call `Make-node('-', p1, p2)` constructs an interior node with the leaves for `x` and `7` as children and we follow the same procedure for pointer `p4` and `p5`, which finally results in `p5` pointing to the root of the constructed syntax tree.

The edges of the syntax tree are shown as solid lines. The underlying parse tree is shown with dotted lines and the dashed lines represent the values of `A.node` and `B.node`, each line points to appropriate node of the syntax tree.

11. What do you understand by syntax-directed translation schemes?

Ans: Syntax-directed translation scheme is an extension of context-free grammar with program fragments (known as **semantic actions**) embedded within production bodies. The semantic actions are generally enclosed within the curly braces, and if the braces are needed as grammar symbols they are put in single quotes. The semantic actions can be placed at any position within a production body. Syntax-directed translation schemes can be considered as a complementary notation to SDD.

Syntax-directed translations can be implemented by first constructing a parse tree and then performing the actions in a left-to-right depth-first order, that is, during a preorder traversal. A syntax-directed translation scheme having both synthesized and inherited attributes needs to be careful while translating them and must follow the given rules:

- ❑ An inherited attribute for a symbol on the right side of a production must be computed in an action before that symbol.
- ❑ A synthesized attribute of a symbol to the right of the action would not be referred by an action.
- ❑ A synthesized attribute for the non-terminal on the left can only be computed after all attributes if references have been computed. The action for computing such attributes can usually be placed at the end of the right side of the production.

Multiple-Choice Questions

1. Which of the following is not true for SDT?
 - (a) It is an extension of CFG.
 - (b) Parsing process is used to do the translation.
 - (c) It does not permit the subroutines to be attached to the production of a CFG.
 - (d) It generates the intermediate code.
2. A parse tree with attribute _____ at each node is known as an annotated parse tree.

(a) Name	(b) Value
(c) Label	(d) None of these
3. Which of the following is true for a dependency graph?
 - (a) The dependency graph helps to determine how the attribute values are computed.
 - (b) It depicts the flow of information among the attribute instances in a parse tree.
 - (c) Both (a) and (b)
 - (d) None of these
4. An SDD is S-attributed if every attribute is _____.

(a) Inherited	(b) Synthesized
(c) Dependent	(d) None of these

5. In L-attributed definitions, the dependency graph edges can go from _____ to _____.
(a) Left to right (b) Right to left
(c) Top to bottom (d) Bottom to top
6. Which of the following is not true for an abstract syntax tree?
(a) It is a compressed form of a parse tree.
(b) It represents the syntactic structure of the source program.
(c) The nodes of the tree represent the operands.
(d) None of these
7. Which of the following is not true for syntax-directed translation schemes?
(a) It is a CFG with program fragments embedded within production bodies.
(b) The semantic actions appear at a fixed position within a production body.
(c) They can be considered as a complementary notation to syntax-directed definitions.
(d) None of these

Answers

1. (c) 2. (b) 3. (c) 4. (b) 5. (a) 6. (c) 7. (b)